

Lab: Infrastructure Decision Analysis - Open Source vs Vendor Solution

Overview

In this comprehensive lab, you will conduct a detailed infrastructure decision analysis for your own project. Your client has accepted your project proposal but has no existing infrastructure. You must compare two infrastructure approaches: an open-source stack (Pinecone, n8n, Hostinger, etc.) versus a vendor solution (Azure, GCP, or AWS).

You will create comprehensive project management deliverables for BOTH approaches, including PESTLE Analysis, Stakeholder Analysis, Project Objectives, Risk Assessment, and Cost Estimation. The lab culminates in a final recommendation based on your analysis.

Client Context:

- Client has accepted your project (use your own project from previous labs/work)
- Client has NO existing infrastructure
- Client has NO in-house DevOps team
- Client HAS a dedicated software operations team (for ongoing maintenance and operations)

Prerequisites: You should have completed previous labs and have a project you've been working on (e.g., client intake automation, proposal generation, workflow automation, etc.).

Estimated Time: 4 hours (full day)

Learning Objectives

After completing this lab, you will be able to:

- Map your current infrastructure stack to equivalent vendor services
- Conduct PESTLE analysis to understand external factors affecting infrastructure decisions
- Perform stakeholder analysis considering infrastructure implications

- Define clear project objectives with Triple Constraint (Quality, Time, Cost) for different infrastructure approaches
- Identify and assess infrastructure-specific and GenAI-specific risks
- Estimate costs and calculate Total Cost of Ownership (TCO) for different infrastructure approaches
- Make data-driven infrastructure recommendations based on comprehensive analysis
- Understand the trade-offs between open-source and vendor solutions

Prerequisites

- A project you've been working on (from previous labs or coursework)
- Understanding of your current infrastructure stack (Pinecone, n8n, Hostinger, or similar)
- Basic knowledge of cloud vendors (Azure, GCP, or AWS)
- Access to vendor pricing information (websites, calculators)
- Understanding of classical project management concepts from the lesson

Instructions

Part 1: Project Selection & Infrastructure Mapping

Time: 30-40 minutes

Before comparing infrastructure approaches, you need to clearly understand your current project and map it to vendor equivalents.

1. Select and document your project

- Choose the project you've been working on (from previous labs or coursework)
- Write a brief project description (2-3 sentences)
- Document the main features and functionality

2. Document current architecture and infrastructure components

- List all infrastructure components you're currently using or planning to use
- Common components might include:
 - **Vector Database:** Pinecone, Weaviate, Qdrant
 - **Workflow Automation:** n8n, Zapier, Make

- **Hosting:** Hostinger, DigitalOcean, Vercel, Netlify
 - **APIs:** OpenAI, Anthropic, Google AI
 - **Storage:** Local storage, cloud storage
 - **Other services:** Monitoring, logging, etc.
- For each component, document:
 - What it does in your project
 - Why you chose it
 - Key features you're using
- 3. **Research and identify equivalent vendor services**
 - Choose ONE vendor to compare against: **Azure, GCP, or AWS** (not all three)
 - Research equivalent services for each of your current components
- 4. **Common Service Mappings:**

For Azure:

 - Pinecone → Azure Cognitive Search / Azure AI Search
 - n8n → Azure Logic Apps
 - Hostinger → Azure App Service / Azure Container Apps
 - OpenAI API → Azure OpenAI Service
- 5. **For AWS:**
 - Pinecone → Amazon Kendra / Amazon OpenSearch Service
 - n8n → AWS Step Functions / AWS AppSync
 - Hostinger → AWS Elastic Beanstalk / AWS App Runner
 - OpenAI API → Amazon Bedrock (Claude) / AWS integration with OpenAI
- 6. **For GCP:**
 - Pinecone → Vertex AI Search / Cloud Search
 - n8n → Cloud Workflows / Cloud Functions
 - Hostinger → Google App Engine / Cloud Run
 - OpenAI API → Vertex AI (Gemini) / GCP integration with OpenAI
- 7. **Compare service capabilities and features**
 - For each mapped service, research:
 - Key features and capabilities
 - How it differs from your current open-source solution

- Advantages and limitations
- Integration requirements

8. Create infrastructure comparison table

- Create a table comparing your current stack to the vendor solution
- Include columns: Current Component, Function, Vendor Equivalent, Key Differences, Notes

Deliverable: Infrastructure comparison table showing your current stack mapped to vendor equivalents, with service mapping and feature comparison notes.

Part 2: PESTLE Analysis for Both Approaches

Time: 40-50 minutes

PESTLE analysis helps you understand external factors that could affect your infrastructure decision. You'll conduct this analysis for BOTH approaches.

1. Research external factors affecting each approach

- Consider how Political, Economic, Social, Technological, Legal, and Ecological factors might differ between open-source and vendor solutions

2. Conduct PESTLE analysis for open-source stack

Create a table with the following structure:

Factor	Open-Source Stack Considerations	Implications
Political		

Economic		
Social		
Technological		
Legal		
Ecological		

3.

Considerations for each factor:

- **Political:** Data sovereignty, government regulations, trade restrictions, open-source policies
- **Economic:** Economic stability, currency fluctuations, local market conditions, cost predictability
- **Social:** Developer community support, talent availability, cultural preferences, open-source adoption
- **Technological:** Technology maturity, innovation pace, compatibility, ecosystem support
- **Legal:** Licensing (open-source licenses), compliance requirements, data protection laws, intellectual property

- **Ecological:** Energy consumption, carbon footprint, sustainability initiatives, green hosting options

4. **Conduct PESTLE analysis for vendor solution**

Create a similar table for your chosen vendor (Azure/GCP/AWS):

Factor	Vendor Solution Considerations	Implications
Political		
Economic		
Social		
Technological		
Legal		

Ecological		
-------------------	--	--

5.

Considerations for each factor:

- **Political:** Vendor location, data residency requirements, government cloud policies, vendor-country relations
- **Economic:** Vendor pricing models, economic stability of vendor, currency exposure, pricing predictability
- **Social:** Vendor reputation, market share, community support, enterprise adoption
- **Technological:** Vendor innovation pace, service maturity, integration capabilities, ecosystem
- **Legal:** Vendor compliance certifications, data processing agreements, SLA terms, liability
- **Ecological:** Vendor sustainability commitments, carbon-neutral regions, green energy usage

6. Compare and document differences

- Create a comparison section highlighting how each PESTLE factor differs between the two approaches
- Note which factors favor which approach
- Document any significant implications for your project

Deliverable: Two comprehensive PESTLE analysis tables (one for open-source stack, one for vendor solution) with detailed comparison notes and implications for each factor.

Part 3: Stakeholder Analysis

Time: 30-40 minutes

Stakeholders are the same for both infrastructure approaches, but their concerns and priorities may differ based on the infrastructure choice.

1. Identify all project stakeholders

- List all stakeholders involved in or affected by your project
- Common stakeholders might include:
 - Client executives/decision-makers
 - Client operations team (your client has a dedicated software operations team)
 - End users
 - Your development team
 - Client IT/security team
 - Client finance/budget team
 - External vendors/partners

2. Analyze stakeholder interests and influence levels

- For each stakeholder, assess:
 - **Interest Level:** How much do they care about this project? (High/Low)
 - **Influence Level:** How much power do they have to affect the project? (High/Low)
 - **Key Concerns:** What matters most to them regarding infrastructure?

3. Create stakeholder quadrant matrix

Create a 2x2 matrix placing stakeholders in quadrants:

4. Copy

High Influence

|

| Quadrant I Quadrant III

| (High Interest, (Low Interest,

| High Influence) High Influence)

|

| Quadrant II Quadrant IV

| (High Interest, (Low Interest,

| Low Influence) Low Influence)

|

|

Low Interest High Interest

5. Explain this code

6. **Example Stakeholder Placement:**

- **Quadrant I (High Interest, High Influence):** Client executives, Client operations team lead
- **Quadrant II (High Interest, Low Influence):** End users, Your development team
- **Quadrant III (Low Interest, High Influence):** Client finance team, Client IT/security team
- **Quadrant IV (Low Interest, Low Influence):** External vendors

7. **Define engagement strategies for each quadrant**

Based on the lesson content (Lines 154-174):

- **Quadrant I (High Interest, High Influence):**
 - Make them members of the steering committee or project team
 - Keep them closely involved
 - Regular updates and decision-making participation
- **Quadrant II (High Interest, Low Influence):**
 - Get them closely involved in the project (e.g., as core project team members)
 - Ensure they feel heard and valued
 - Regular communication and feedback loops
- **Quadrant III (Low Interest, High Influence):**
 - Consider making them members of the consultant panel or advisory board
 - Regularly and proactively involve them in the communication process
 - Keep them informed to maintain support
- **Quadrant IV (Low Interest, Low Influence):**
 - Provide summarized information on a situational basis
 - Monitor for changes in interest or influence

8. **Consider infrastructure-specific stakeholder concerns**

- How does infrastructure choice affect each stakeholder?

- Open-source concerns: Technical complexity, maintenance burden, learning curve
- Vendor concerns: Vendor lock-in, cost predictability, dependency on vendor
- Document how infrastructure choice might change stakeholder priorities

Deliverable: Stakeholder analysis matrix with quadrant placement, engagement plan for each quadrant, and infrastructure-specific considerations documented.

Part 4: Define Project Objectives for Both Approaches

Time: 40-50 minutes

Project objectives may differ between infrastructure approaches. You'll define clear objectives for each approach following the criteria from the lesson.

1. Review project requirements and constraints

- Review your project description
- Consider client constraints (no DevOps, but has operations team)
- Identify key success criteria

2. Create objectives for open-source approach

Following the lesson criteria (Lines 175-194), objectives should be:

- **Precise** - Specific and unambiguous
- **Comprehensible** - Clear to all stakeholders
- **Positive** - Stated in affirmative terms
- **Quantifiable** - Measurable where possible
- **Solution Neutral** - Don't close off potential solutions prematurely

3. Example Objectives for Open-Source Approach:

- "Deploy the application using self-hosted infrastructure within 8 weeks"
- "Maintain system uptime of 99% during business hours"
- "Keep infrastructure costs below \$500/month for the first year"
- "Enable the operations team to maintain the system with minimal external support"

- "Ensure all data remains within client-controlled infrastructure"

4. **Create objectives for vendor solution approach**

Example Objectives for Vendor Solution:

- "Deploy the application on [Azure/GCP/AWS] within 6 weeks using managed services"
- "Maintain system uptime of 99.9% with vendor SLA guarantees"
- "Scale infrastructure automatically based on usage patterns"
- "Enable the operations team to manage the system through vendor console with minimal technical expertise"
- "Ensure compliance with [vendor] certifications (SOC 2, ISO 27001, etc.)"

5. **Define Triple Constraint for each approach**

For each approach, clearly define:

Quality:

- What are the quality requirements?
- Performance metrics, reliability, security standards
- Example: "Response time < 2 seconds for 95% of requests"

6. Time:

- What is the time framework?
- Milestones, deadlines, delivery schedule
- Example: "Deploy MVP within 6 weeks, full system within 12 weeks"

7. Cost:

- What are the budget constraints?
- Total budget, cost per month, cost per user
- Example: "Total infrastructure cost not to exceed \$10,000 in first year"

8. **Compare how objectives differ**

- Create a comparison table showing how objectives differ between approaches
- Note which approach might prioritize speed vs. cost vs. control
- Document trade-offs in objectives

Deliverable: Two comprehensive project objectives documents (one for each approach) with Triple Constraint definitions (Quality, Time, Cost) and detailed comparison showing how objectives differ.

Part 5: Risk Assessment for Both Approaches

Time: 50-60 minutes

Infrastructure choices come with different risks. You'll conduct comprehensive risk assessments for both approaches, including GenAI-specific risks.

1. Conduct risk identification workshop for open-source stack

Focus on infrastructure-specific risks:

- **Maintenance burden:** Ongoing updates, patches, monitoring
- **Technical complexity:** Requires DevOps expertise (but client has no DevOps)
- **Vendor lock-in avoidance:** Flexibility but more responsibility
- **Scalability challenges:** Manual scaling, capacity planning
- **Security responsibility:** Client is responsible for security
- **Talent requirements:** Need skilled team members
- **Documentation and support:** Relies on community support

2. Also include GenAI-specific risks from the lesson (Lines 445-498):

- Model failure or degradation
- Data quality issues
- User misuse
- Model choice errors
- Bias and fairness concerns
- Cost overruns from unexpected usage
- Model updates breaking functionality

3. Conduct risk identification for vendor solution

Focus on vendor-specific risks:

- **Vendor lock-in:** Difficult to migrate away
- **Cost overruns:** Usage-based pricing can be unpredictable
- **Dependency on vendor:** Service outages, API changes
- **Compliance issues:** Vendor compliance may not match client needs
- **Service availability:** Vendor outages affect your system
- **Limited customization:** May not support all use cases
- **Vendor policy changes:** Terms, pricing, service changes

4. Also include GenAI-specific risks:
- Same GenAI risks as above, but consider vendor-specific aspects
 - Vendor model updates affecting your application
 - Vendor API rate limits or changes
5. **Assess probability and impact for each risk**
- For each identified risk, assess:
- **Probability:** High / Medium / Low
 - **Impact:** High / Medium / Low
 - Create a risk matrix/heat map for visualization
6. **Risk Matrix Example:**

7. Copy

Impact

High | Critical Critical High

| Risks Risks Risks

Med | High Medium Low

| Risks Risks Risks

Low | **Medium** Low Low

| Risks Risks Risks

|

Low **Medium** High

Probability

8. Explain this code

9. **Define mitigation strategies for each risk**

Use the strategies from the lesson (Lines 483-498):

- **Avoidance:** Eliminate the risk entirely
- **Reduction:** Reduce the probability (evaluation, monitoring, human in the loop, red teaming)
- **Mitigation:** Reduce the impact (roll back, disable features)
- **Transfer:** Move risk to another party (contractual agreements, insurance)
- **Acceptance:** Monitor without active mitigation if risk is low enough

10. For each risk, document:

- Risk description
- Probability and Impact
- Mitigation strategy

- Specific mitigation actions
- Responsible party

11. Create risk registers for both approaches

Format for risk register:

Frequency	Risk Description	Probability	Impact	Mitigation Strategy	Mitigation Actions	Responsible Party
High	[Risk description]	High / Medium / Low	High / Medium / Low	[Strategy]	[Actions]	

12.

Compare risks between approaches

- Create a comparison highlighting which risks are unique to each approach
- Note which approach has higher overall risk
- Document how risks differ in nature (e.g., technical vs. business risks)

Deliverable: Two comprehensive risk registers/matrices (one for open-source stack, one for vendor solution) with detailed mitigation plans, probability/impact assessments, and comparison notes.

Part 6: Cost Estimation for Both Approaches

Time: 50-60 minutes

Cost estimation is critical, especially for GenAI projects. You'll create detailed cost breakdowns and calculate TCO for both approaches.

1. Research current pricing for open-source stack components

Research and document costs for:

- Self-hosting infrastructure (servers, VPS, cloud instances)
- Maintenance and operations time
- Developer time for setup and ongoing support
- Variable costs (API usage, compute, storage)
- Fixed costs (hosting, licenses, infrastructure)

2. Break down costs for open-source stack

Use the GenAI cost categories from the lesson (Lines 409-416):

1. Data Costs:

- Data cleaning, chunking, embedding
- Storage costs
- Data processing compute

3. 2. Model Costs:

- Experimentation with models
- RAG setup and maintenance
- Prompt engineering time
- API costs (OpenAI, Anthropic, etc.)

4. 3. Application Costs:

- API integrations
- Orchestration (n8n hosting, workflow execution)
- Application hosting

5. 4. Evaluation Costs:

- LLM as judges (API calls for evaluation)
- Human review time
- Testing infrastructure

6. **5. Ops Costs:**

- Monitoring tools
- Logging and analytics
- Cost tracking tools
- Operations team time

7. **6. Compliance Costs:**

- Audits
- PII filtering
- Guardrails implementation
- Security tools

8. **Cost Estimation Formula (from lesson, Lines 427-435):**

Rough Estimate: Month inference cost + Total build cost

Monthly Inference Cost: (avg. prompt tokens + avg. completion tokens) × calls/month × \$/token

Total Build Cost: (corpus size in documents × embedding cost/doc) + (dev person-months × day rate) + (monthly infra cost × projected months to launch)

9. **Research vendor pricing (Azure/GCP/AWS)**

Use vendor pricing calculators and documentation:

- Base subscription costs and tiers
- Managed service fees
- Usage-based pricing models
- Reserved instances vs. pay-as-you-go
- Data transfer costs
- Storage costs

10. **Break down costs for vendor solution**

Use the same GenAI cost categories, but map to vendor services:

1. Data Costs:

- Vendor data processing services
- Storage (vendor storage services)
- Data transfer costs

11. **2. Model Costs:**

- Vendor AI/ML services (Azure OpenAI, AWS Bedrock, GCP Vertex AI)

- Model training/fine-tuning costs
- API usage costs

12.3. Application Costs:

- Vendor workflow services (Logic Apps, Step Functions, Cloud Workflows)
- Application hosting (App Service, Elastic Beanstalk, App Engine)
- API gateway costs

13.4. Evaluation Costs:

- Vendor AI services for evaluation
- Monitoring and analytics services

14.5. Ops Costs:

- Vendor monitoring services (Azure Monitor, CloudWatch, Cloud Monitoring)
- Logging services
- Cost management tools

15.6. Compliance Costs:

- Vendor compliance certifications (included or additional)
- Security services
- Audit tools

16. Calculate TCO (Total Cost of Ownership) for both approaches

Calculate TCO over different time periods:

- **Year 1:** Initial setup + first year operations
- **Year 2-3:** Ongoing operations
- **Total 3-year TCO:** Compare long-term costs

17. Include hidden costs:

- Training costs (team training on new infrastructure)
- Migration costs (if switching later)
- Downtime costs
- Opportunity costs

18. Create cost comparison

Create a side-by-side comparison showing:

- Cost breakdown by category
- Monthly costs

- Annual costs
- 3-year TCO
- Cost assumptions and variables
- Cost risk factors (what could cause overruns)

Deliverable: Two detailed cost estimation documents (one for each approach) with:

- Cost breakdown by GenAI category (Data, Model, Application, Evaluation, Ops, Compliance)
- Variable and fixed costs clearly separated
- TCO calculations for 1-year and 3-year periods
- Cost comparison table
- Assumptions and risk factors documented

Part 7: Final Recommendation

Time: 30-40 minutes

Synthesize all your analysis into a final recommendation for your client.

1. Create comparison matrix across all criteria

Create a comprehensive comparison table:

Criteria	Open-Source Stack	Vendor Solution	Winner
PESTLE Factors	[Summary]	[Summary]	[Which?]

Stakeholder Alignment	[Summary]	[Summary]	[Which?]
Project Objectives	[Summary]	[Summary]	[Which?]
Risk Profile	[Summary]	[Summary]	[Which?]
Cost (Year 1)	\$(Amount)	\$(Amount)	[Which?]
Cost (3-Year TCO)	\$(Amount)	\$(Amount)	[Which?]
Time to Deploy	[Timeframe]	[Timeframe]	[Which?]
Operations Complexity	[Level]	[Level]	[Which?]

2.

Synthesize findings from all previous parts

- Review your PESTLE analysis - which factors favor which approach?
- Review stakeholder analysis - which approach better addresses stakeholder concerns?
- Review objectives - which approach better meets project objectives?
- Review risks - which approach has more manageable risks?
- Review costs - which approach is more cost-effective short-term and long-term?

3. **Make your recommendation**

- Choose: **Open-Source Stack OR Vendor Solution**
- Be decisive - don't recommend "both" or "it depends"

4. **Document comprehensive decision rationale**

Explain WHY you're making this recommendation:

- What were the key deciding factors?
- How does this align with client constraints (no DevOps, but has operations team)?
- What are the primary benefits of your chosen approach?
- What risks are you accepting with this choice?

5. **Identify key trade-offs**

- What are you sacrificing by choosing this approach?
- What would you gain with the other approach?
- Why are the trade-offs acceptable?

6. **Address implementation considerations**

- What are the next steps if client accepts this recommendation?
- What is the implementation timeline?
- What resources are needed?
- What are the critical success factors?

Deliverable: Comprehensive recommendation document including:

- Executive summary (1-2 paragraphs)
- Comparison matrix across all criteria
- Clear recommendation (open-source OR vendor solution)

- Detailed decision rationale
- Key trade-offs analysis
- Implementation considerations and next steps

Deliverables

Required Deliverables

Submit all of the following deliverables. Each deliverable should be clearly labeled and organized.

1. Infrastructure Comparison Table

- Infrastructure comparison table showing your current stack
- Mapping of each component to vendor equivalent service
- Feature comparison notes for each mapped service
- Rationale for choosing your vendor (Azure, GCP, or AWS)

Format: Table or document with clear sections for current stack, vendor equivalents, and comparisons.

2. PESTLE Analysis (Both Approaches)

Open-Source Stack PESTLE:

- Complete PESTLE analysis table for open-source stack
- All six factors addressed (Political, Economic, Social, Technological, Legal, Ecological)
- Implications documented for each factor

Vendor Solution PESTLE:

- Complete PESTLE analysis table for vendor solution
- All six factors addressed
- Implications documented for each factor

Comparison:

- Comparison notes highlighting differences between approaches
- Identification of which factors favor which approach

Format: Two tables (one for each approach) plus comparison section.

3. Stakeholder Analysis

- Complete list of all project stakeholders
- Stakeholder quadrant matrix (2x2) with all stakeholders placed
- Engagement strategies defined for each quadrant (I, II, III, IV)
- Infrastructure-specific stakeholder considerations documented
- Notes on how infrastructure choice affects stakeholder concerns

Format: Matrix/diagram plus engagement plan document.

4. Project Objectives (Both Approaches)

Open-Source Stack Objectives:

- Project objectives document for open-source approach
- Objectives follow criteria: Precise, Comprehensible, Positive, Quantifiable, Solution Neutral
- Triple Constraint defined: Quality (with metrics), Time (with milestones), Cost (with budget)
- Minimum 3-5 clear objectives

Vendor Solution Objectives:

- Project objectives document for vendor solution approach
- Objectives follow same criteria
- Triple Constraint defined: Quality, Time, Cost
- Minimum 3-5 clear objectives

Comparison:

- Comparison table or section showing how objectives differ
- Trade-offs in objectives documented

Format: Two documents (one for each approach) plus comparison section.

5. Risk Assessment (Both Approaches)

Open-Source Stack Risks:

- Comprehensive risk register for open-source stack
- Minimum 8-10 risks identified

- Each risk includes: Description, Probability, Impact, Mitigation Strategy, Mitigation Actions
- Infrastructure-specific risks included
- GenAI-specific risks included (from lesson)
- Risk matrix/heat map visualization

Vendor Solution Risks:

- Comprehensive risk register for vendor solution
- Minimum 8-10 risks identified
- Each risk includes: Description, Probability, Impact, Mitigation Strategy, Mitigation Actions
- Vendor-specific risks included
- GenAI-specific risks included
- Risk matrix/heat map visualization

Comparison:

- Comparison of risk profiles
- Notes on which approach has higher/lower risk
- Notes on how risk types differ

Format: Two risk registers (tables) plus risk matrices plus comparison section.

6. Cost Estimation (Both Approaches)

Open-Source Stack Costs:

- Cost breakdown by GenAI category (Data, Model, Application, Evaluation, Ops, Compliance)
- Variable costs clearly identified and estimated
- Fixed costs clearly identified and estimated
- Monthly cost estimate
- Annual cost estimate (Year 1)
- 3-year TCO calculated
- Cost assumptions documented
- Cost risk factors identified

Vendor Solution Costs:

- Cost breakdown by GenAI category (mapped to vendor services)
- Variable costs clearly identified and estimated
- Fixed costs clearly identified and estimated
- Monthly cost estimate
- Annual cost estimate (Year 1)
- 3-year TCO calculated
- Cost assumptions documented
- Cost risk factors identified

Comparison:

- Side-by-side cost comparison table
- TCO comparison (1-year and 3-year)
- Cost analysis and insights

Format: Two cost estimation documents plus comparison table. Use formulas from lesson where applicable.

7. Final Recommendation

- Executive summary (1-2 paragraphs)
- Comprehensive comparison matrix across all criteria (PESTLE, Stakeholders, Objectives, Risk, Cost)
- Clear recommendation: **Open-Source Stack** OR **Vendor Solution** (choose one)
- Detailed decision rationale explaining WHY
- Key trade-offs analysis (what you're sacrificing)
- Implementation considerations:
 - Next steps if recommendation is accepted
 - Implementation timeline
 - Required resources
 - Critical success factors
- Addresses client constraints (no DevOps, but has operations team)

Format: Professional recommendation document (2-3 pages) suitable for presenting to client.

Submission Format

- All deliverables should be submitted as a single document or organized folder
- Use clear headings and sections
- Include tables, matrices, and visualizations where helpful
- Ensure all deliverables are complete and clearly labeled
- Professional formatting suitable for client presentation

Evaluation Criteria

Your work will be evaluated on:

- **Completeness:** All deliverables completed for both approaches
- **Depth of Analysis:** Thorough research and detailed analysis in each section
- **Quality of Comparison:** Clear, meaningful comparisons between approaches
- **Decision Quality:** Well-reasoned recommendation with clear rationale
- **Professionalism:** Clear communication, proper formatting, suitable for client presentation
- **Alignment with Lesson:** Proper application of project management concepts from the lesson

Good luck with your infrastructure decision analysis!